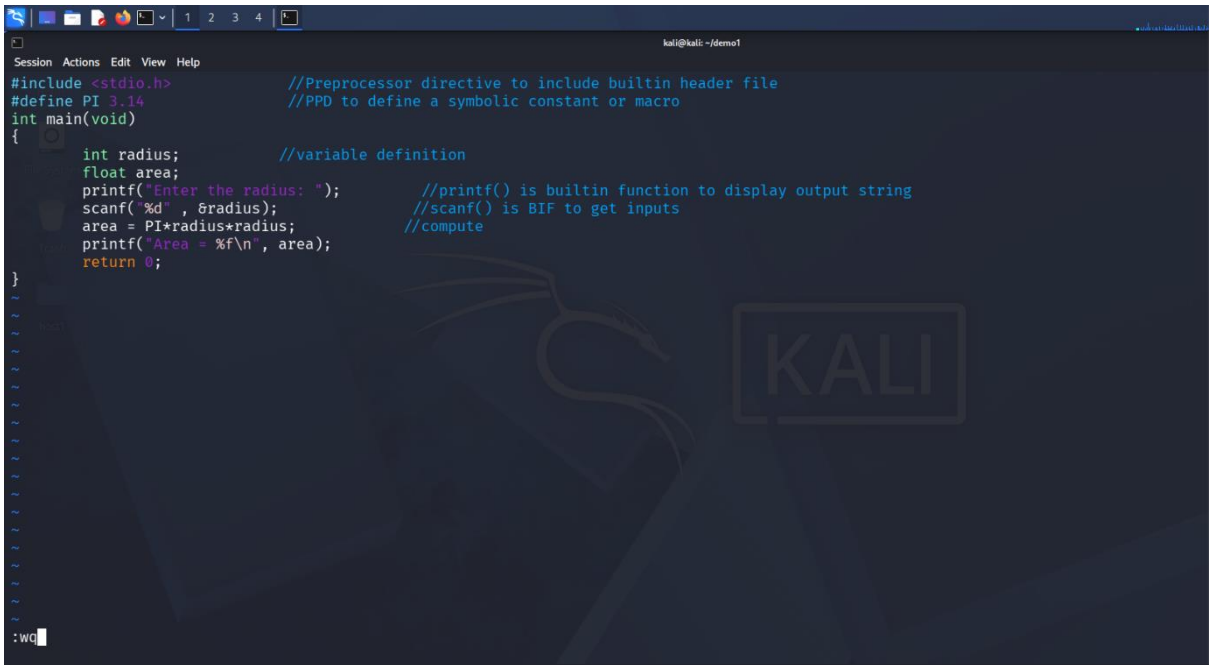
Experiment-2

Demonstration of Stages of Compilation

- **Preprocessing Stage:** Preprocessing is the **first stage** of compilation. The preprocessor handles commands beginning with #, such as #include and #define. It expands header files, replaces macros, and removes comments from the source code.
  For example, #include <stdio.h> is processed and the required contents of the header file are included.
  Input: area.c
  Output: area.i



- **Compilation Stage**: In this stage, the compiler converts the pre-processed C code into assembly language. It checks the program for syntax and other compilation-related errors.
  The generated assembly code contains low-level instructions that are closer to the machine language understood by the processor.

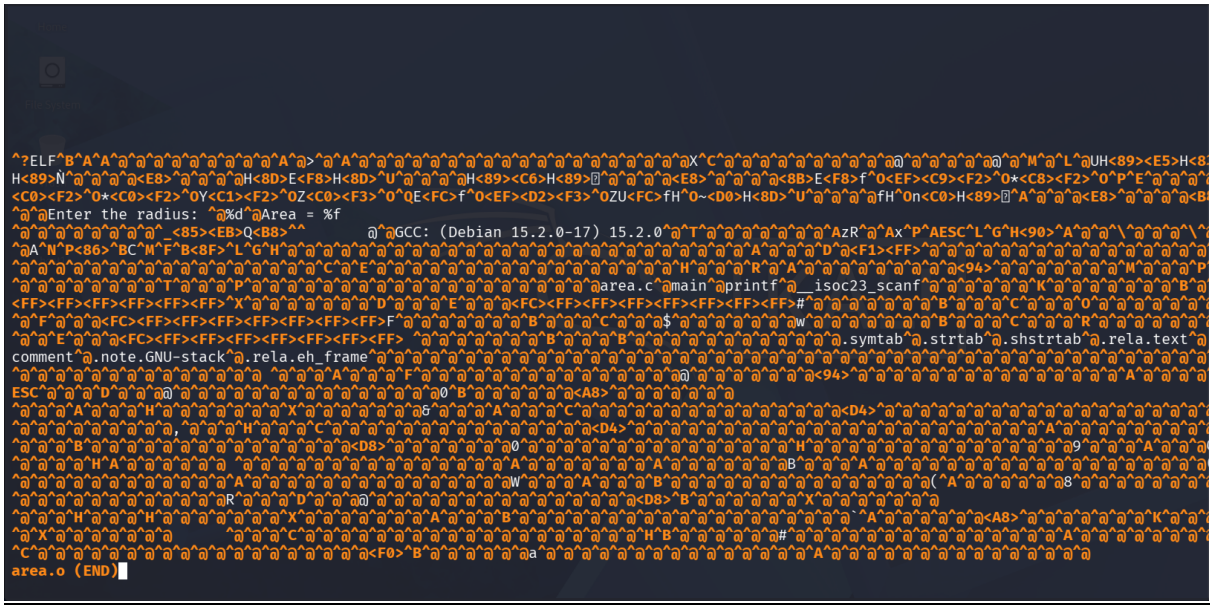
Input: circle.i
Output: circle.s

```
Session  Actions  Edit  View  Help
# 0 "area.c"
# 0 "<built-in>"
# 0 "<command-line>"
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 0 "<command-line>" 2
# 1 "area.c"
# 1 "/usr/include/stdio.h" 1 3 4
# 28 "/usr/include/stdio.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 1 3 4
# 33 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 3 4
# 1 "/usr/include/features.h" 1 3 4
# 415 "/usr/include/features.h" 3 4
# 1 "/usr/include/features-time64.h" 1 3 4
# 20 "/usr/include/features-time64.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 21 "/usr/include/features-time64.h" 2 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 1 3 4
# 19 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 20 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 2 3 4
# 22 "/usr/include/features-time64.h" 2 3 4
# 416 "/usr/include/features.h" 2 3 4
# 523 "/usr/include/features.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 1 3 4
# 730 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 731 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/long-double.h" 1 3 4
# 732 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
# 524 "/usr/include/features.h" 2 3 4
# 547 "/usr/include/features.h" 3 4
area.i
```

- **<u>Assembly Stage:</u>** The **assembler** converts the assembly language code into **machine-level object code**. This code is stored in an object file.
  The object file contains machine instructions but is not yet a complete executable program because some functions and libraries may still need to be connected.
  Input: circle.s
  Output: circle.o

```
        .file   "area.c"
        .text
        .section        .rodata
.LC0:
        .string "Enter the radius: "
.LC1:
        .string "%d"
.LC3:
        .string "Area = %f\n"
        .text
        .globl  main
        .type   main, @function
main:
.LFB0:
        .cfi_startproc
        pushq   %rbp
        .cfi_def_cfa_offset 16
        .cfi_offset 6, -16
        movq    %rsp, %rbp
        .cfi_def_cfa_register 6
        subq    $16, %rsp
        leaq    .LC0(%rip), %rax
        movq    %rax, %rdi
        movl    $0, %eax
        call    printf@PLT
        leaq    -8(%rbp), %rax
        leaq    .LC1(%rip), %rdx
        movq    %rax, %rsi
        movq    %rdx, %rdi
        movl    $0, %eax
        call    __isoc23_scanf@PLT
area.s
```

- **Linking Stage:** The linker combines the object file with the required libraries and other necessary program components. For example, functions such as printf() and scanf() are provided through standard libraries.
  After linking is completed, a final executable file is created.
  Input: circle.o + required libraries
  Output: circle



- **Executing Stage:** In the final stage, the operating system loads the executable program into main memory (RAM). The processor then executes the machine instructions.
  For the circle-area program, the user enters the radius, and the program calculates and displays the area. Radius taken is 5

- **circle.c**
  ↓
  Preprocessing
  ↓
  circle.i
  ↓
  Compilation
  ↓
  circle.s
  ↓
  Assembly

↓

circle.o

↓

Linking

↓

circle (Executable)

↓

Execution

↓

Output

Submitted BY: Reyansh Pandit

SAP ID:590033602

Batch:16